

Bash Function Fundamentals: Building Blocks for Shell Script Automation

Function

- In Bash scripting, a function is a reusable block of code that performs a specific task.
- Functions allow you to organize and modularize your code, making it easier to read, maintain, and debug.
- Here's a structured guide from basic to advanced, with practical examples for each level:

1. Basics of Functions in Bash

Objective: Introduce the syntax and purpose of functions in Bash.

Syntax:

```
function_name() {  
    # Commands  
}  
  
OR  
  
function function_name {  
    # Commands  
}
```

Example:

```
greet() {  
    echo "Hello, $1!"  
}  
  
# Calling the function  
greet "Aamer" # Output: Hello, Aamer!
```

Explanation:

Functions are defined using **function_name() { }**. They can be called multiple times and allow parameters (**\$1, \$2, etc.**).

2. Function Parameters and Return Values

Objective: Teach students to pass arguments and capture return values from functions.

The University of Lahore

Passing Parameters:

```
add() {  
    result=$(( $1 + $2 ))  
    echo $result  
}  
  
# Call the function with arguments  
sum=$(add 3 5)  
echo "The sum is $sum" # Output: The sum is 8
```

Returning Values:

Since Bash functions can't return values directly like other languages, we use **echo** or global variables.

```
multiply() {  
    echo $(( $1 * $2 ))  
}  
  
product=$(multiply 4 7)  
echo "Product is $product" # Output: Product is 28
```

Explanation:

Parameters are accessed using \$1, \$2, etc., and return values are handled by capturing **echo** output.

3. Using Local Variables in Functions

Objective: Prevent variable conflicts by using local variables.

Syntax and Example:

```
calculate_square() {  
    local num=$1  
    local result=$(( num * num ))  
    echo $result  
}  
  
# Call the function  
square=$(calculate_square 6)  
echo "Square is $square" # Output: Square is 36
```

The University of Lahore

Explanation:

local restricts the scope of a variable to the function, avoiding interference with variables outside the function.

Function Scope and Local Variables:

- Understand the concept of function scope and variable visibility.
- Use the `local` keyword to declare variables within a function.
- Demonstrate the difference between local and global variables.

Example:

```
# Function scope and local variables
function print_message() {
    local message="Hello from the function!"
    echo "$message"
}
message="Hello from the main script!"
print_message
echo "$message"
```

4. Advanced Function Reusability with Sourcing

Objective: Reuse functions across multiple scripts by sourcing.

Creating a Function Library (`my_functions.sh`):

```
# Define reusable functions
greet() {
    echo "Hello, $1!"
}
add() {
    echo $(( $1 + $2 ))
}
```

Using the Function Library:

```
# Source the function library
source my_functions.sh
# Call functions from the library
```

The University of Lahore

```
greet "Aamer"                # Output: Hello, Aamer!  
sum=$(add 10 15)  
echo "Sum is $sum"          # Output: Sum is 25
```

Explanation:

By using `source my_functions.sh`, all functions from `my_functions.sh` are available in the current script, promoting modularity.

5. Recursive Functions

Objective: Implement recursive logic in Bash functions.

Example (Factorial):

```
factorial() {  
    if [ $1 -le 1 ]; then  
        echo 1  
    else  
        local temp=$(( $1 - 1 ))  
        local result=$(factorial $temp)  
        echo $(( $1 * result ))  
    fi  
}  
  
# Call the recursive function  
fact=$(factorial 5)  
echo "Factorial is $fact"  # Output: Factorial is 120
```

Explanation:

This factorial function calls itself, demonstrating recursion. Recursive functions require a base case to avoid infinite loops.

6. Using Functions with Arrays

Objective: Pass arrays to functions and work with complex data structures.

Example:

```
print_array() {  
    local arr=("$@")  
    for element in "${arr[@]}; do
```

The University of Lahore

```
        echo "$element"
    done
}
# Define an array and pass it to the function
numbers=(1 2 3 4 5)
print_array "${numbers[@]}"
```

Explanation:

Arrays are passed using "\$@" to handle each element properly, allowing functions to process lists of items.

7. Advanced Modularization and Function Namespaces

Objective: Use naming conventions to avoid conflicts in large scripts or when sourcing multiple function files.

Example:

```
math_add() {
    echo $(( $1 + $2 ))
}
math_subtract() {
    echo $(( $1 - $2 ))
}
# Usage
sum=$(math_add 8 3)
diff=$(math_subtract 10 5)
echo "Sum: $sum, Difference: $diff"
```

Explanation:

By prefixing functions (e.g., `math_`), you create namespaces, reducing the chance of conflicts and making the script more readable.

8. Error Handling and Debugging in Functions

Objective: Implement error handling to make functions more robust.

Example:

```
divide() {
```

The University of Lahore

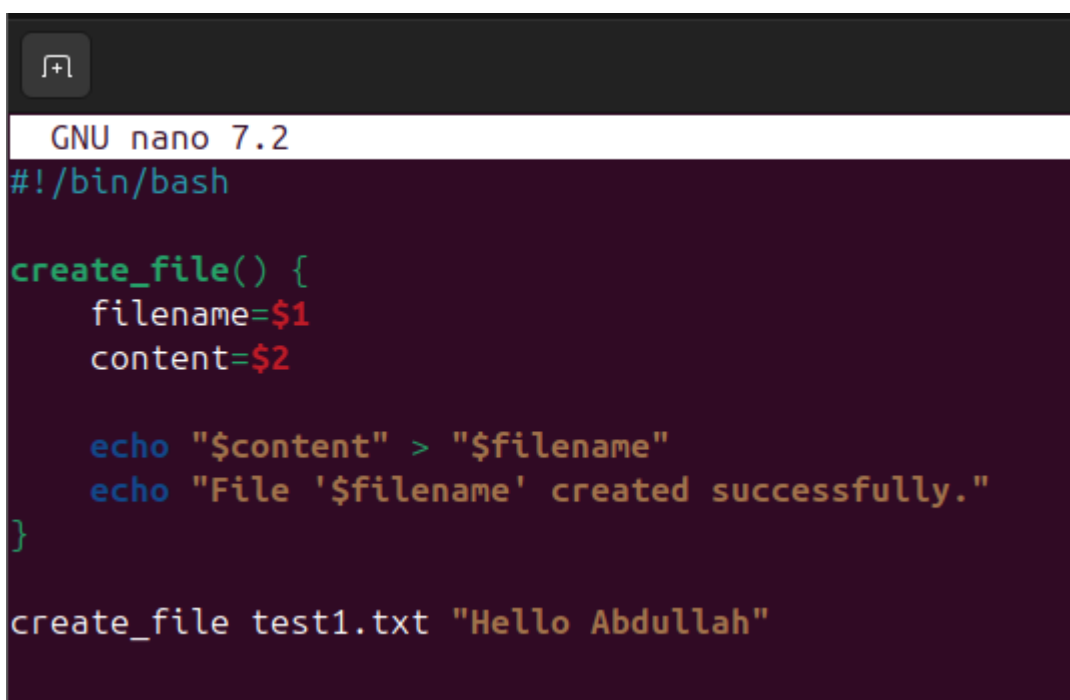
```
if [ $2 -eq 0 ]; then
    echo "Error: Division by zero"
    return 1
else
    echo $(( $1 / $2 ))
fi
}
result=$(divide 10 0)
if [ $? -ne 0 ]; then
    echo "An error occurred."
else
    echo "Result is $result"
fi
```

Explanation:

Functions can handle errors by checking conditions and using `return` to indicate failure, with `$?` capturing the exit status.

Practice Questions

- 1) Write a function `create_file` that takes a filename and some content as arguments, creates the file (if it doesn't exist), and writes the content into the file.



```
GNU nano 7.2
#!/bin/bash

create_file() {
    filename=$1
    content=$2

    echo "$content" > "$filename"
    echo "File '$filename' created successfully."
}

create_file test1.txt "Hello Abdullah"
```

The University of Lahore

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano create_file.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x create_file.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./create_file.sh
File 'test1.txt' created successfully.
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ cat test1.txt
Hello Abdullah
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

- 2) Write a function `append_to_file` that takes a filename and some content, appends the content to the file (creating it if it doesn't exist), and notifies the user.

```
GNU nano 7.2
#!/bin/bash

append_to_file() {
    filename=$1
    content=$2

    echo "$content" >> "$filename"
    echo "Content appended to '$filename'."
}

append_to_file test1.txt "This is appended line"

abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano append_to_file.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x append_to_file.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./append_to_file.sh
Content appended to 'test1.txt'.
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

- 3) Write a function `read_file` that reads the contents of a file and prints it to the terminal.

The University of Lahore

```
GNU nano 7.2
#!/bin/bash

read_file() {
    filename=$1

    if [ -f "$filename" ]; then
        cat "$filename"
    else
        echo "File does not exist."
    fi
}

read_file test1.txt

abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano read_file.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x read_file.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./read_file.sh
Hello Abdullah
This is appended line
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

- 4) Write a function `check_file_exists` that checks if a file exists and returns an appropriate message.

```
GNU nano 7.2
#!/bin/bash

check_file_exists() {
    filename=$1

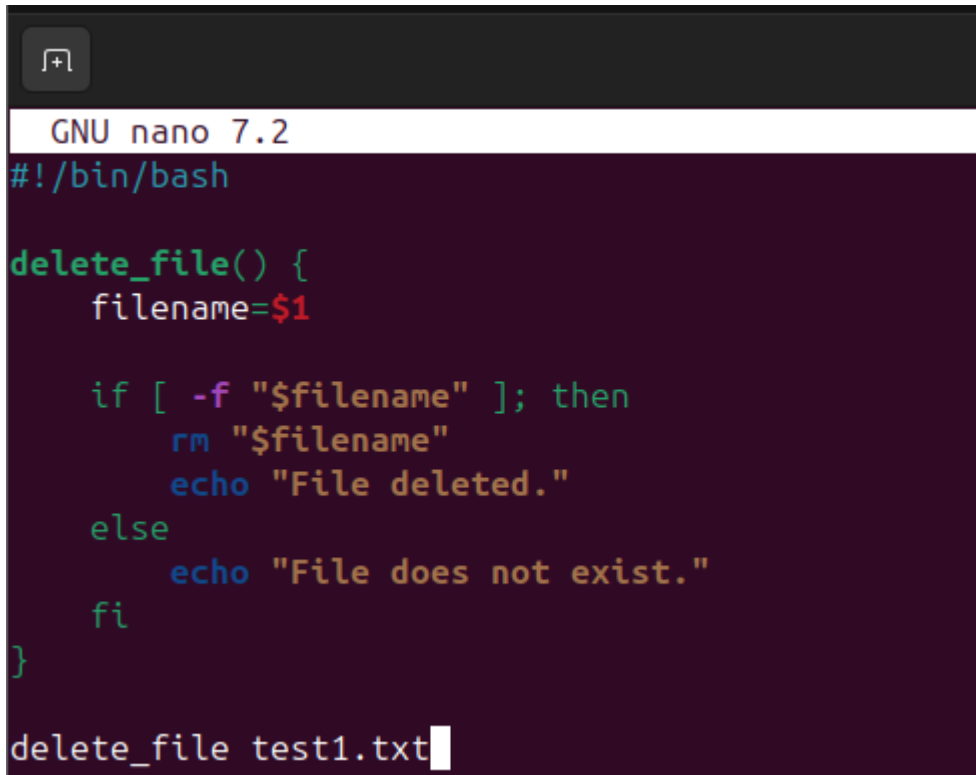
    if [ -f "$filename" ]; then
        echo "File exists."
    else
        echo "File does not exist."
    fi
}

check_file_exists test1.txt
```


The University of Lahore

```
File exists.
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano check_file_exists.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x check_file_exists.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./check_file_exists.sh
File exists.
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

- 5) Write a function `delete_file` that deletes a specified file and notifies the user of the action.



```
GNU nano 7.2
#!/bin/bash

delete_file() {
    filename=$1

    if [ -f "$filename" ]; then
        rm "$filename"
        echo "File deleted."
    else
        echo "File does not exist."
    fi
}

delete_file test1.txt
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano delete_file.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x delete_file.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./delete_file.sh
File deleted.
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ cat test1.txt
cat: test1.txt: No such file or directory
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

- 6) Write a function `backup_file` that creates a backup of a file by copying it with a `.bak` extension.

The University of Lahore

```
GNU nano 7.2
#!/bin/bash

backup_file() {
    filename=$1

    if [ -f "$filename" ]; then
        cp "$filename" "$filename.bak"
        echo "Backup created."
    else
        echo "File does not exist."
    fi
}

backup_file test1.txt
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano backup_file.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x backup_file.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./backup_file.sh
File does not exist.
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./backup_file.sh
Backup created.
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

- 7) Write a function `count_lines` that counts and prints the number of lines in a given file.

```
GNU nano 7.2
#!/bin/bash

count_lines() {
    filename=$1

    if [ -f "$filename" ]; then
        wc -l "$filename"
    else
        echo "File does not exist."
    fi
}

count_lines test1.txt
```

The University of Lahore

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano count_lines.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x count_lines.sh
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./count_lines.sh
1 test1.txt
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```

- 8) Write a function `search_word` that searches for a specific word in a file and prints all the lines containing that word.

```
GNU nano 7.2
#!/bin/bash

search_word() {
    filename=$1
    word=$2

    if [ -f "$filename" ]; then
        grep "$word" "$filename"
    else
        echo "File does not exist."
    fi
}

search_word test1.txt "Hello"
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ nano search_word.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ chmod +x search_word.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$ ./search_word.sh
Hello Abdullah
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~$
```